

Smytten E-Commerce Database Management System

CHAPTER – 7

ORDERS AND ORDER_DETAILS ANALYSIS

4.1 Introduction

The Smytten E-Commerce Database Management System manages customer orders and the products included in each order. The Orders table stores general information about an order, such as the customer who placed the order, order date, total amount, and order status. The Order_Details table stores the individual products included in each order along with their quantities and prices.

The two tables are related using Order_ID. This structure avoids repetition of order information and allows multiple products to be associated with a single order.

4.2 ORDERS TABLE

Table Structure

Attribute	Description	Key
Order_ID	Unique identification of the order	Primary Key
Customer_ID	Customer who placed the order	Foreign Key
Order_Date	Date on which order was placed	-
Total_Amount	Total amount of the order	-
Order_Status	Current status of the order	-

4.3 ORDER_DETAILS TABLE

Table Structure

Attribute	Description	Key
Order_Detail_ID	Unique identification of order detail	Primary Key
Order_ID	Identifies the order	Foreign Key
Product_ID	Product included in the order	Foreign Key
Quantity	Number of products ordered	-
Price	Price of the product at the time of order	-

4.4 Relationship Between Orders and Order_Details

The relationship between the two entities is:

Orders 1: M Order_Details

One order can contain multiple products, while each order-detail record belongs to one order.

CUSTOMER

|

| 1: M

↓

ORDERS

|

| 1: M

↓

ORDER_DETAILS

|

| M: 1

↓

PRODUCT

4.5 Sample Orders Records

For your Smytten project, we can prepare **10 Orders records**.

Order_ID	Customer_ID	Order_Date	Total_Amount	Order_Status
201	101	10-09-2026	899	Delivered
202	102	11-09-2026	1299	Shipped
203	103	11-09-2026	599	Delivered
204	104	12-09-2026	1499	Pending
205	105	13-09-2026	799	Delivered
206	106	14-09-2026	1099	Shipped
207	107	15-09-2026	699	Pending
208	108	16-09-2026	1599	Delivered

Order_ID	Customer_ID	Order_Date	Total_Amount	Order_Status
209	109	17-09-2026	999	Shipped
210	110	18-09-2026	749	Pending

4.6 Sample Order_Details Records

Order_Detail_ID	Order_ID	Product_ID	Quantity	Price
301	201	101	2	400
302	201	102	1	99
303	202	103	2	650
304	203	104	1	599
305	204	105	2	749
306	205	101	1	799
307	206	102	2	550
308	207	103	1	699
309	208	104	2	799
310	209	105	1	999

4.7 Order Insertion Operation

A new order can be inserted into the Orders table using an INSERT statement.

```
SQL> INSERT INTO ORDERS
  2  VALUES (211, 101, SYSDATE, 899, 'Pending');

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL> INSERT INTO ORDER_DETAILS
  2  VALUES (311, 211, 101, 2, 400);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

4.8 Order Modification Operation

The order status can be modified when the order progresses.

```
SQL> UPDATE ORDERS
  2   SET ORDER_STATUS = 'Shipped'
  3   WHERE ORDER_ID = 211;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL> UPDATE ORDER_DETAILS
  2   SET QUANTITY = 3
  3   WHERE ORDER_DETAIL_ID = 311;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.
```

4.9 Customer Order History Report

The customer order history can be generated by joining Customer, Orders, Order_Details, and Product tables.

```
SQL> SELECT
  2     C.CUSTOMER_NAME,
  3     O.ORDER_ID,
  4     O.ORDER_DATE,
  5     P.PRODUCT_NAME,
  6     OD.QUANTITY,
  7     OD.PRICE,
  8     O.TOTAL_AMOUNT,
  9     O.ORDER_STATUS
10  FROM CUSTOMER2 C
11  JOIN ORDERS O
12     ON C.CUSTOMER_ID = O.CUSTOMER_ID
13  JOIN ORDER_DETAILS OD
14     ON O.ORDER_ID = OD.ORDER_ID
15  JOIN PRODUCT2 P
16     ON OD.PRODUCT_ID = P.PRODUCT_ID
17  ORDER BY C.CUSTOMER_NAME, O.ORDER_DATE;
```

CUSTOMER_NAME				
ORDER_ID		ORDER_DAT		
PRODUCT_NAME				
QUANTITY	PRICE	TOTAL_AMOUNT	ORDER_STATUS	
Ananya				
202	11-SEP-26			
Lipstick				
2	650	1299	Shipped	
CUSTOMER_NAME				
ORDER_ID		ORDER_DAT		
PRODUCT_NAME				
QUANTITY	PRICE	TOTAL_AMOUNT	ORDER_STATUS	
Dhivya				
203	11-SEP-26			
Mascara				
1	599	599	Delivered	

This report displays the customer's name, order ID, order date, purchased product, quantity, price, total order amount, and order status.

4.10 Conclusion

The Orders and Order_Details tables provide an organized method for managing customer purchases in the Smytten E-Commerce Database Management System. The Orders table stores general order information, while the Order_Details table stores the products and quantities associated with each order. The relationship between these tables reduces data redundancy and maintains data consistency. Order insertion, modification, and customer order history reports help manage and analyze customer transactions efficiently.